Лабораторная работа № 4. Связи моделей. Внешний вид шаблона

Цель работы: Освоить создание и настройку связей между моделями Django (ForeignKey, ManyToManyField, OneToOneField), изучить параметры связей (on_delete, related_name, null, blank, default), научиться применять миграции при добавлении связей к существующим моделям, а также настраивать отображение связанных данных в административной панели Django.

Рассмотрим связи моделей, которые описываются в Django полями отношений (смотрите документацию https://django.fun/ru/docs/django/4.1/ref/models/fields/) (рисунок 1).

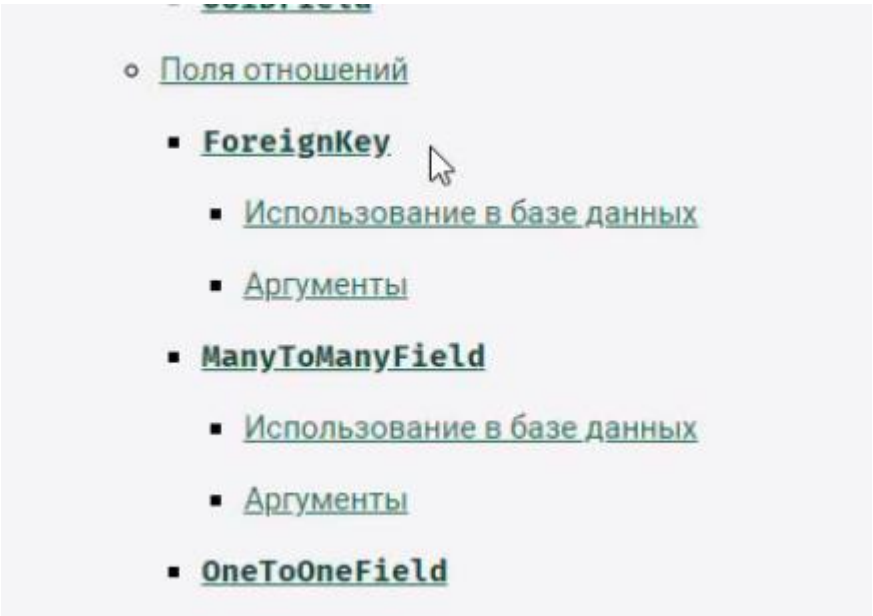


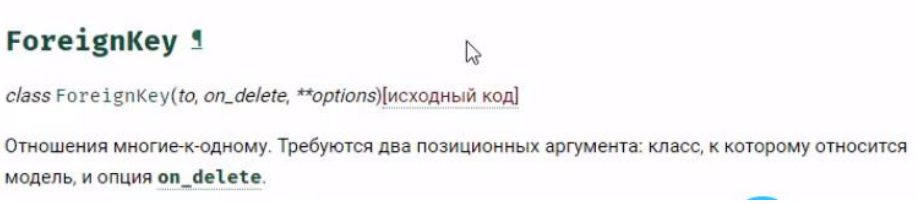
Рисунок 1

Как известно из теории баз данных, данные могут храниться в различных таблицах. Можно все данные хранить в одной таблице, но тогда нарушаются принципы нормализации данных. Например, рассмотрим наш пример с новостным сайтом. Как правило, все новости можно разбить на категории. И можно было бы создать в таблице news это дополнительное поле Категории новости и в нем писать название новости, которая относится к этой категории. Но опят, мы здесь бы нарушали принцип нормализации данных.

Поэтому нужно создать еще одну табличку, в которой будет храниться категория. А в таблице news, будет храниться номер категории, к которой относиться данная новость. Таким образом, мы избегаем следующей проблемы, что если нам для чего-то нужно будет в какой-то момент переименовать категорию, то это можно сделать только один раз в самой таблице Категорий, а в самой таблице news, как был когда-то назначено значение внешнего ключа, так оно и останется не тронутым, не смотря на изменения.

Таким образом, мы понимаем, что связанные данные могут храниться в различных таблицах.

Для этого Django предлагает три специальных класса для организации связей.



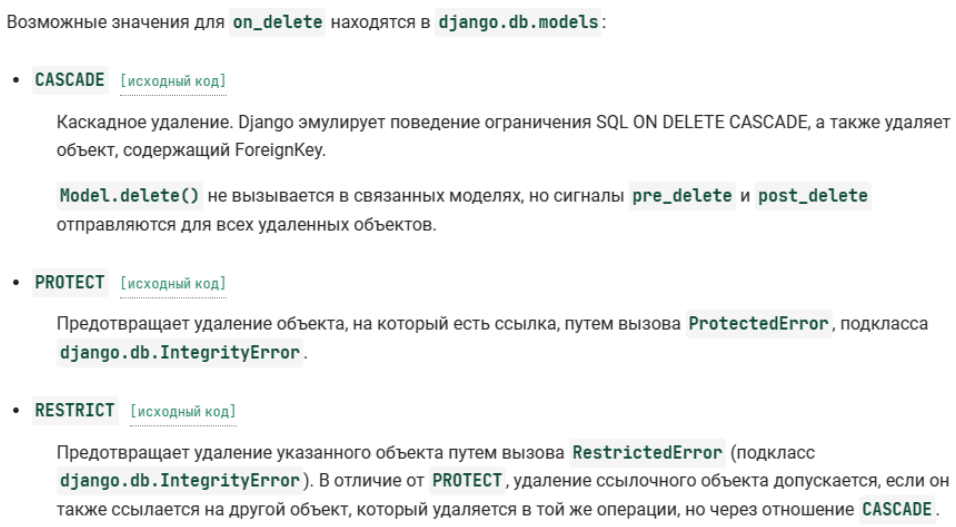
1 категория – много новостей

Нам необходимо объявить поле типа ForeignKey - отношения многие-к-одному.

Опция on_delete позволяет организовать ограничения «Что делать с зависимым объектом при удалении объекта, на который он ссылается.

Например, есть категория Культура и с ней связано много новостей. Если вдруг удалили категорию Культуру, то вторым аргументов on_delete необходимо определить, что делать в этом случае.

В качестве значений on_delete можно указать различные виды поведений (рисунок 2)



Возможные значения для `on_delete` находятся в `django.db.models`:

- `CASCADE` [исходный код]

  Каскадное удаление. Django эмулирует поведение ограничения SQL ON DELETE CASCADE, а также удаляет объект, содержащий ForeignKey.

  `Model.delete()` не вызывается в связанных моделях, но сигналы `pre_delete` и `post_delete` отправляются для всех удаленных объектов.

- `PROTECT` [исходный код]

  Предотвращает удаление объекта, на который есть ссылка, путем вызова `ProtectedError`, подкласса `django.db.IntegrityError`.

- `RESTRICT` [исходный код]

  Предотвращает удаление указанного объекта путем вызова `RestrictedError` (подкласс `django.db.IntegrityError`). В отличие от `PROTECT`, удаление ссылочного объекта допускается, если он также ссылается на другой объект, который удаляется в той же операции, но через отношение `CASCADE`.

Рисунок 2

Рассмотрим тип ForeignKey.

Перейдем к нашей модели и создадим новую модель Категории. Модель Категория первичная, Новости – вторичная

db_index=True – этот параметр необходим для индексирования этого поля

Индексация позволяет организовать более быстрый поиск. Как вы знаете, СУБД поле primary key является индексированным по умолчанию (рисунок 3).

class Category(models.Model):

title=models.CharField(max_length=150, db_index=True, verbose_name='**Наименование категории**')

```
class Category(models.Model):
    title=models.CharField(max_length=150, db_index=True, verbose_name='Наименование категории')
```

Рисунок 3

И также создадим подкласс Meta (рисунок 4):

```
class Category(models.Model):
    title=models.CharField(max_length=150, db_index=True, verbose_name='Наименование категории')

    class Meta:
        verbose_name='Категория'
        verbose_name_plural = 'Категории'
        ordering=['title']# сортировка по наименованию названия
```

Рисунок 4

Можно сделать миграцию, конечно. Но для начала, необходимо связать эти модели и для этого воспользуемся классом ForeignKey. Он записывается во вторичной модели как поле

Ссылку на модель необходимо указано, до того, как объявлена первичная модель. Потому первичную модель необходимо написать раньше. Т.е. перенесем ее над классом news (рисунок 5)

```
from django.db import models

class Category(models.Model):
    title=models.CharField(max_length=150, db_index=True, verbose_name='Наименование категории')

    class Meta:
        verbose_name='Категория'
        verbose_name_plural = 'Категории'
        ordering=['title']# сортировка по наименованию названия


class News(models.Model):
    title=models.CharField(max_length=150, verbose_name='Наименование')
    content=models.TextField(blank=True, verbose_name='Контент')
    created_at=models.DateTimeField(auto_now_add=True, verbose_name='Дата публикации')
    updated_at=models.DateTimeField(auto_now=True, verbose_name='Обновлено')
    photo=models.ImageField(upload_to='photos/%Y/%m/%d', verbose_name='Фото', blank=True)
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey()

    def __str__(self):
        return self.title

    class Meta:
        verbose_name='Новость'
        verbose_name_plural = 'Новости'
        ordering=['-created_at',]
```

Рисунок 5

И теперь можем изменить модель news (рисунок 6, 7, 8).

```
class News(models.Model):
    title=models.CharField(max_length=150, verbose_name='Наименование')
    content=models.TextField(blank=True, verbose_name='Контент')
    created_at=models.DateTimeField(auto_now_add=True, verbose_name='Дата публикации')
    updated_at=models.DateTimeField(auto_now=True, verbose_name='Обновлено')
    photo=models.ImageField(upload_to='photos/%Y/%m/%d', verbose_name='Фото', blank=True)
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey(Category)
```

Рисунок 6

Но если класс был объявлен позже, то этот параметр класса ForeignKey, должен быть указан в виде строки (рисунок 7)

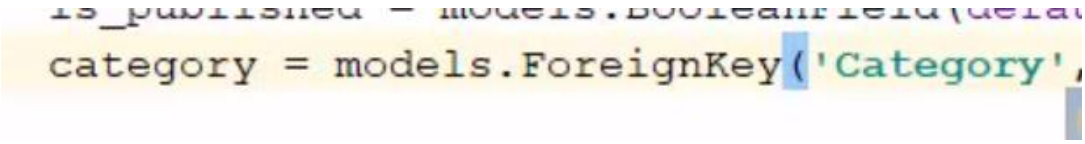


Рисунок 7

```
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey(Category, on_delete=models.PROTECT())
```

Рисунок 8

Не забудьте прописать второй аргумент класса ForeignKey on_delete.

При вводе функции PROTECT в данном случае необходимо удалить скобки (). В соответствии с документацией не мы вызываем эту функцию с параметрами, а Django сам вызовет ее в нужный момент. Мы ее просто прописываем.

Итак, models.PROTECT обеспечивает защиту удаления связанных данных,

Далее, известно, что все поля по умолчанию прописываются как обязательные для заполнения. При этом в нашей модели уже есть записи, и мы ее связываем с моделью Категории с помощью ForeignKey. В связи с этим СУБД будет строить какие-то ограничения и поэтому добавить новое обязательное поле в модель, которая имеет уже записи уже так просто не получится. СУБД не позволит нам это сделать. Поэтому мы воспользуемся опцией NULL (рисунок 9)

```
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey(Category, on_delete=models.PROTECT, null=True)
```

Рисунок 9

Этот параметр позволит нам не заполнять это поле.

Допишем еще параметр default=1, который обозначает, что по умолчанию поле заполнится 1.

```
    updated_at=models.DateTimeField(auto_now=True, verbose_name='Обновлено')
    photo=models.ImageField(upload_to='photos/%Y/%m/%d', verbose_name='Фото', blank=True)
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey(Category, on_delete=models.PROTECT, null=True, default=1)
```

Рисунок 10

Теперь перейдем к созданию и выполнению миграции (рисунок 11).

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py makemigrations
Migrations for 'news':
  news\migrations\0003_category_news_category.py
    - Create model Category
    - Add field category to news

(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>
```

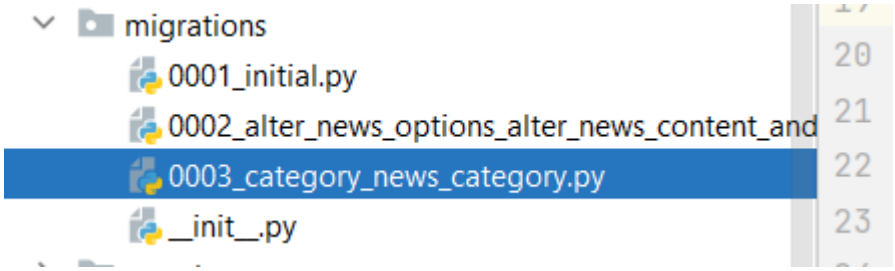


Рисунок 11

Теперь выполним миграцию (рисунок 12)

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py migrate
Operations to perform:
```

Рисунок 12

При этом у нас насыпались ошибки (рисунок 13)

```
  with self.connection.schema_editor(
File "C:\Users\Елена\PycharmProjects\pythonProject29\venv\Lib\site-packages\django\db\backends\sqlite3\schema.py", line 38, in __exit__
  self.connection.check_constraints()
File "C:\Users\Елена\PycharmProjects\pythonProject29\venv\Lib\site-packages\django\db\backends\sqlite3\base.py", line 264, in check_constraints
  raise IntegrityError(
django.db.utils.IntegrityError: The row in table 'news_news' with primary key '8' has an invalid foreign key: news_news.category_id contains a value '1' that does not
have a corresponding value in news_category.id.

(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>
```

Рисунок 13

Это произошло потому, что у нас нет в таблице Категории такого идентификатора с 1.

Это нарушает целостность данных. И как результат возникновение этой ошибки.

Удаляем default=1 (рисунок 14)

```
is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
category=models.ForeignKey(Category, on_delete=models.PROTECT, null=True, default=1)
```

```
    is_published=models.BooleanField(default=True, verbose_name='Опубликовано')
    category=models.ForeignKey(Category, on_delete=models.PROTECT, null=True)
```

Рисунок 14

Снова создаем миграцию с номером 0004 (рисунок 15).

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py makemigrations
Migrations for 'news':
  news\migrations\0004_alter_news_category.py
    - Alter field category on news
```

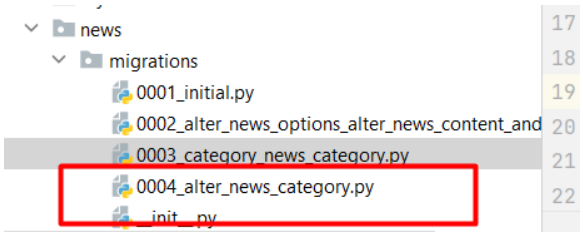


Рисунок 15

Снова делаем миграцию и у нас снова ошибка (рисунок 16)

```
    raise IntegrityError(
django.db.utils.IntegrityError: The row in table 'news_news' with primary key '8' has an invalid foreign key: news_news.category_id contains a value '1' that does not
 have a corresponding value in news_category.id.
```

Рисунок 16

Это получилось так, потому что у нас есть миграция 003, которую Django так же послушно ее выполняет, которая содержит default=1

Посмотрим файл этой третьей миграции (рисунок 17)

```
),
migrations.AddField(
    model_name='news',
    name='category',
    field=models.ForeignKey(default=1, null=True, on_delete=django.db.models.deletion.PROTECT, to='news.categor
),
```

Рисунок 17

Поэтому удаляем 003 и 004 миграции и повторяем еще раз этапы миграции (рисунок 18).

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py makemigrations
Migrations for 'news':
  news\migrations\0003_category_news_category.py
    - Create model Category
    - Add field category to news
```

Рисунок 18

И как видим, у нас получилось выполнить миграции. Перезапустим БД (рисунок 19).

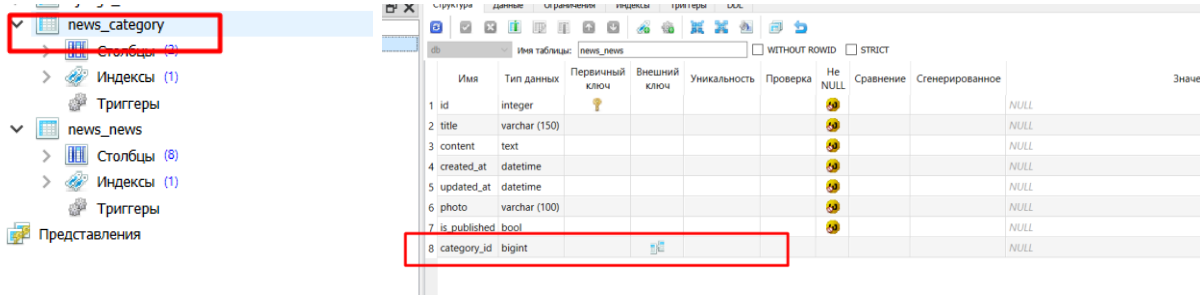


Рисунок 19

Как мы видим, в таблице появилось новое поле Категория (рисунок 20).

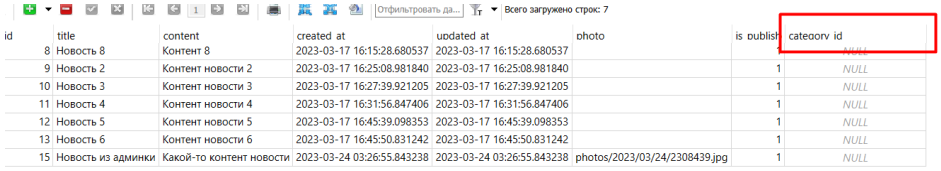
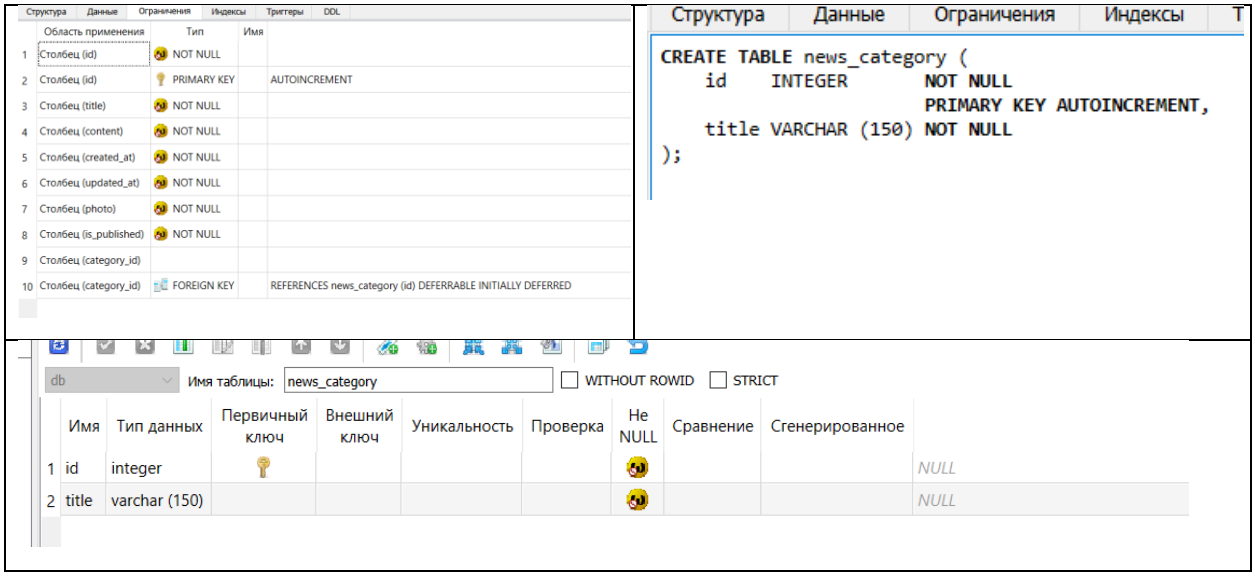


Рисунок 20

В ограничениях мы видим уже прописанные ограничения по полю Категории id первичный ключ (рисунок 21).



```
CREATE TABLE news_news (
    id           INTEGER       NOT NULL
                               PRIMARY KEY AUTOINCREMENT,
    title        VARCHAR (150) NOT NULL,
    content      TEXT          NOT NULL,
    created_at   DATETIME      NOT NULL,
    updated_at   DATETIME      NOT NULL,
    photo        VARCHAR (100) NOT NULL,
    is_published BOOL          NOT NULL,
    category_id  BIGINT        REFERENCES news_category (id) DEFERRABLE INITIALLY DEFERRED
);
```

Рисунок 21

Разберемся, что необходимо еще сделать, чтобы эта связь заработала.

По сути дела, уже все готово для работы, только можно настроить внешний вид админки.

Первое, это надо зарегистрировать модель в файле admin.py (рисунок 22)

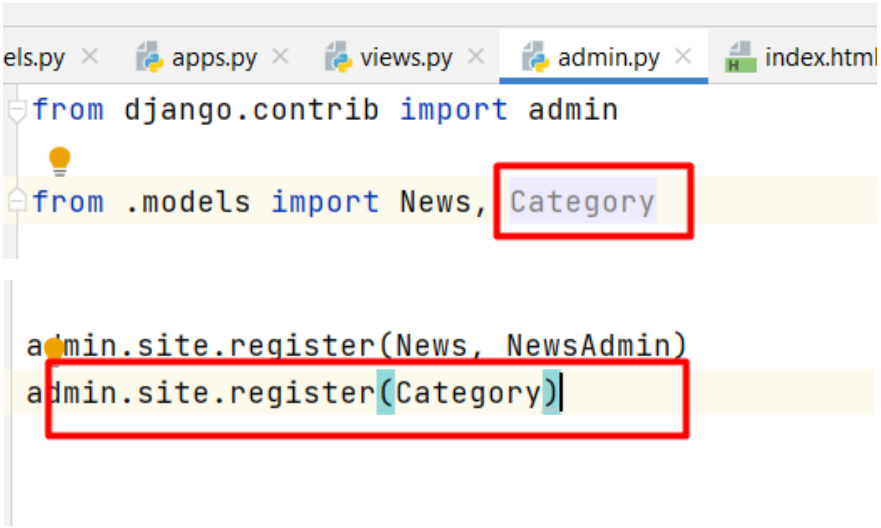


Рисунок 23

И настроим (рисунок 24)

```
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('id', 'title')
    list_display_links =('id','title')
    search_fields =('title',)# здесь аргументом является кортеж, поэтому в конце ставим запятую,
    # указывя, что остальных значений картежа нет


admin.site.register(News, NewsAdmin)
admin.site.register(Category, CategoryAdmin)
```

Рисунок 24

Обновим админку в браузере (рисунок 25)

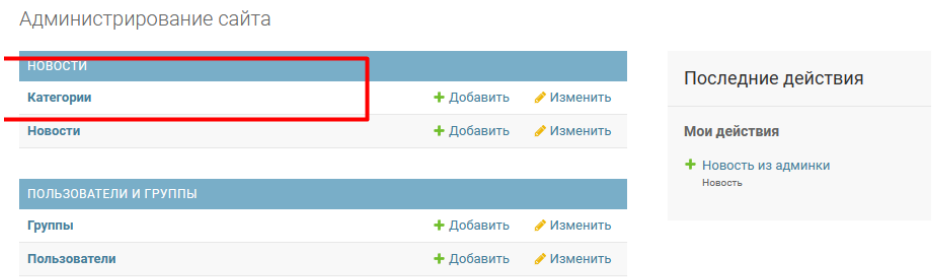


Рисунок 25

Добавим категории (категория 26).



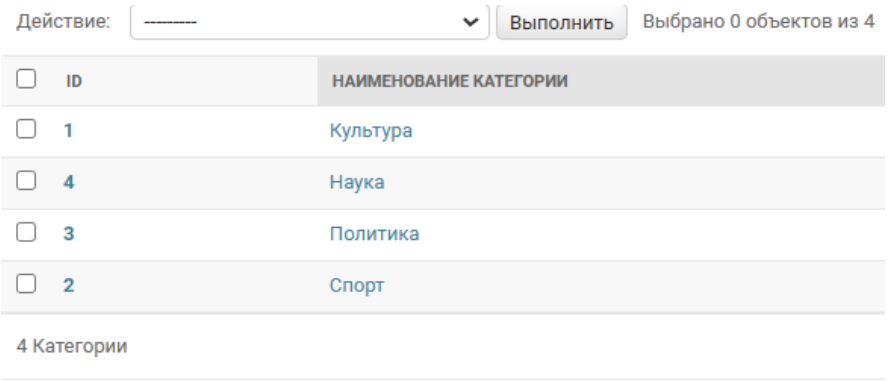
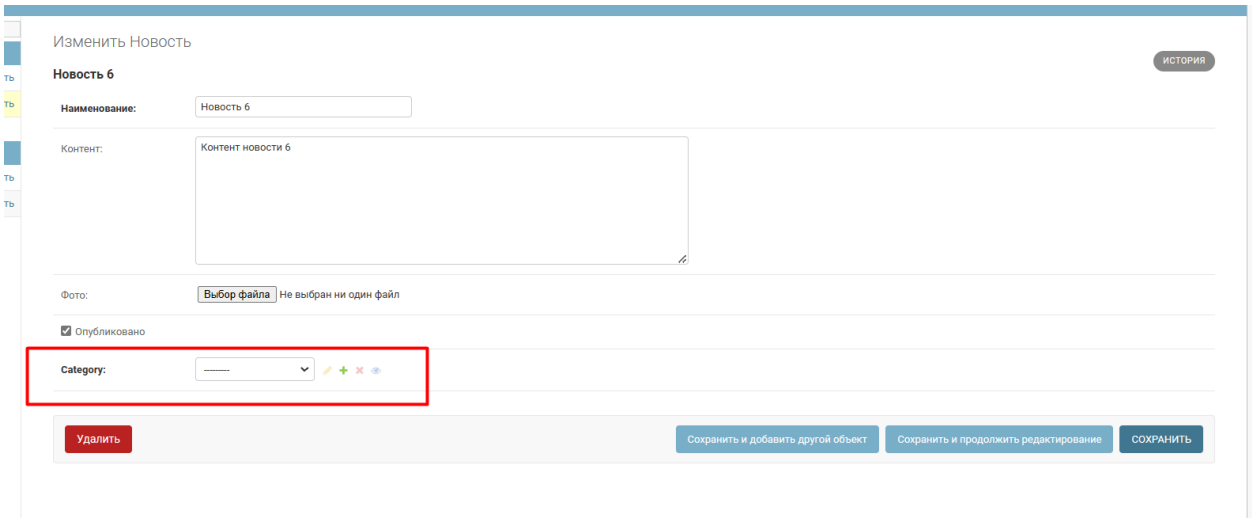


Рисунок 26

Перейдем в Новости (рисунок 27)



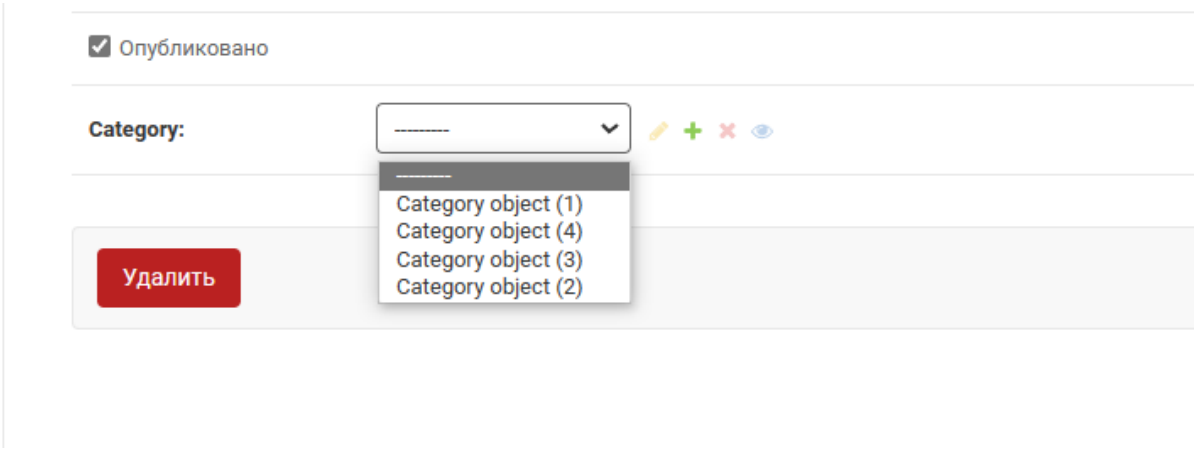


Рисунок 27

Давайте уберем непонятный object().

Для этого добавим строгое представление объекта (рисунок 28).

```
from django.db import models

class Category(models.Model):
    title=models.CharField(max_length=150, db_index=True, verbose_name='Ha

    def __str__(self):
        return self.title

    class Meta:
        verbose_name='Категория'
        verbose_name_plural = 'Категории'
        ordering=['title']# сортировка по наименованию названия
```

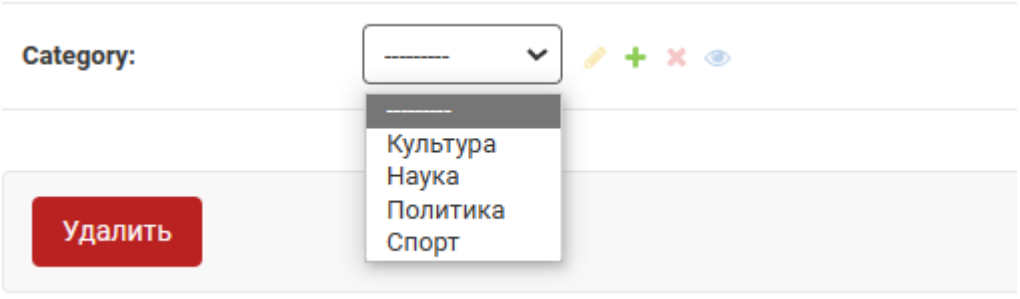


Рисунок 28

Попытаемся Сохранить, но будет ошибка (рисунок 29)

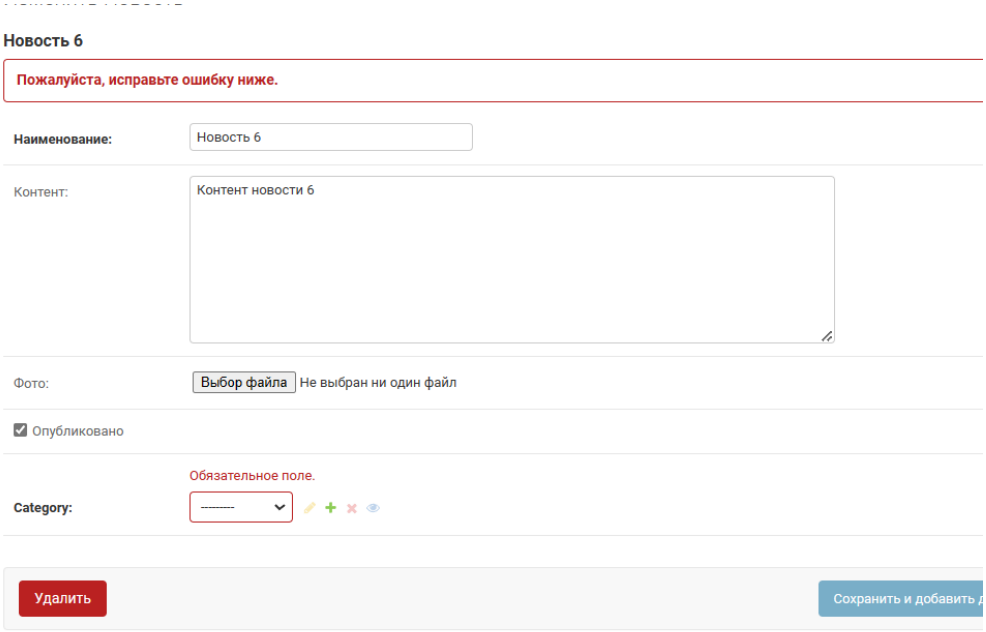


Рисунок 29

Потому что, оно обязательное для валидатора.

И тогда нужно добавить blank=True

Но это мы делать не будем, а просто выберем категории и заполним поле.

Пройдем по всем новостям и присвоим им категории (рисунок 30).

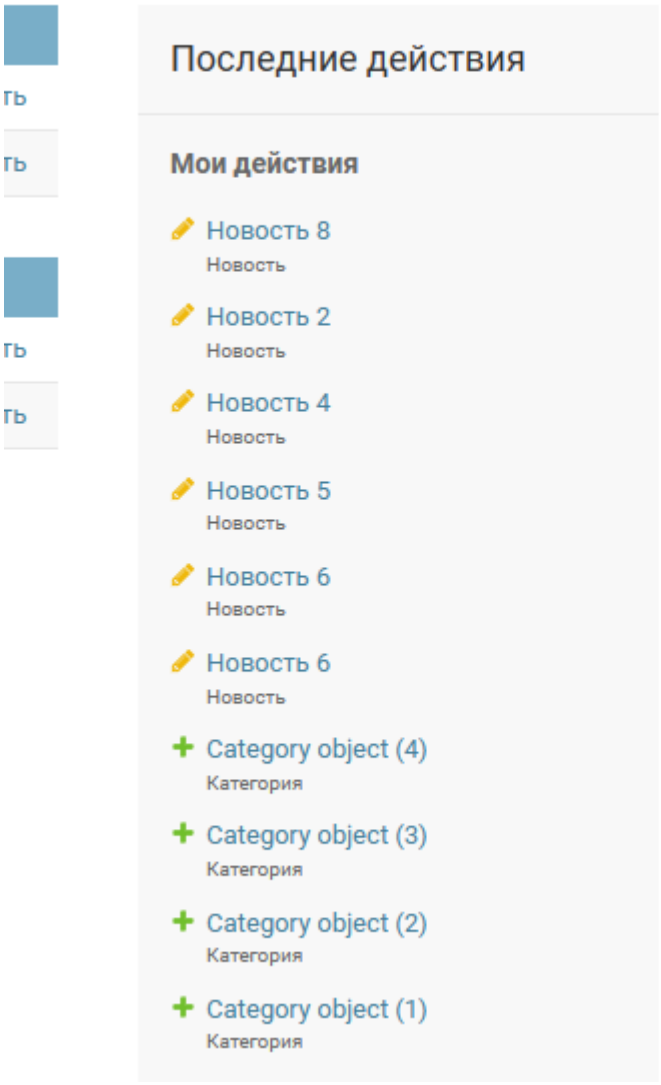


Рисунок 30

Настроим админку в модели Новости выводилась категория (рисунок 31)

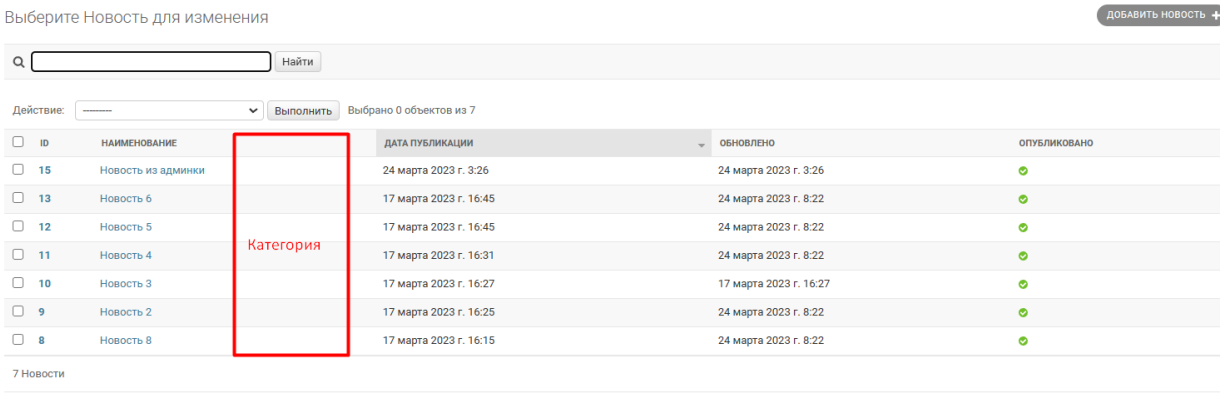


Рисунок 32

```python
class NewsAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'category', 'created_at', 'updated_at','is_published')
    list_display_links =('id','title')
    search_fields =('title','content')

class CategoryAdmin(admin.ModelAdmin):
```

Рисунок 32

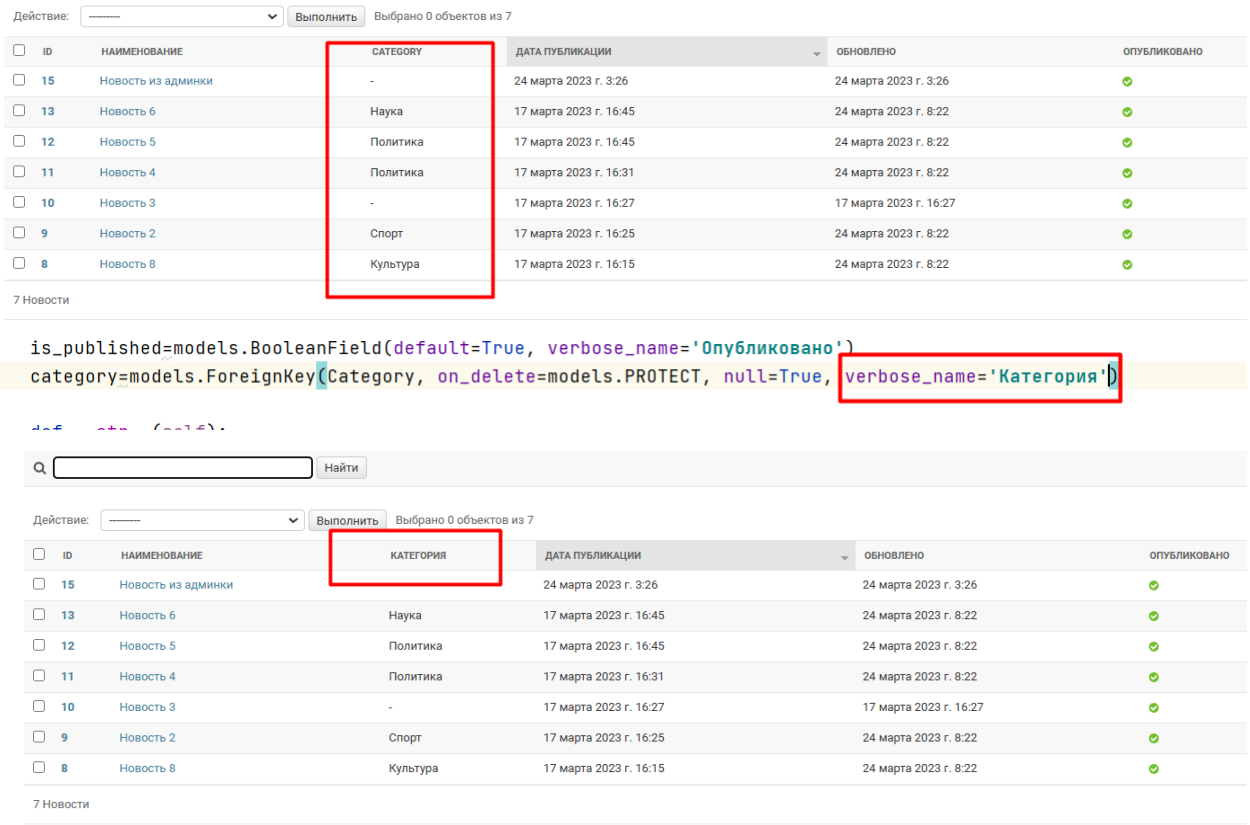


Рисунок 33

Давайте еще улучшим нашу админку. Сделаем редактируемые поля и добавим фильтры. Например, в поле is_published сделать возможность установки из поля админки (рисунок 34).

```python
class NewsAdmin(admin.ModelAdmin):
    list_display = ('id', 'title', 'category', 'created_at', 'updated_at','
    list_display_links =('id','title')
    search_fields =('title','content')
    list_editable = ('is_published',)
    list_filter = ('is_published', 'category')
```

Рисунок 34

Проверим результаты настройки (рисунок 35)

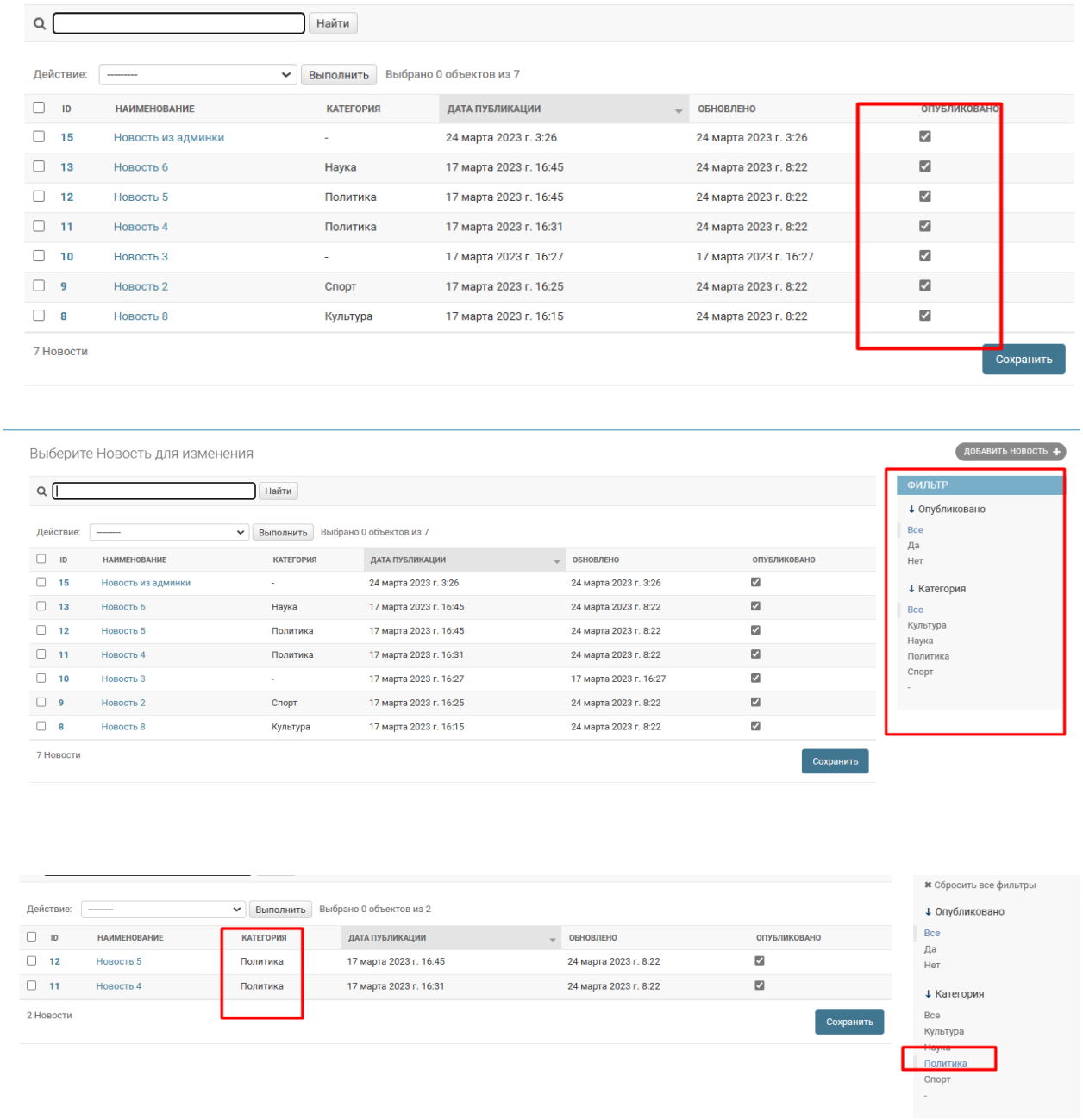


Рисунок 35

Индивидуальные задания

Общие требования для всех вариантов

Для каждого варианта необходимо:

1. Создать две или более связанных моделей в Django.
2. Применить соответствующее поле связи (ForeignKey, ManyToManyField или OneToOneField).

3. Настроить параметры связи (on_delete, related_name, null, blank, default).
4. Выполнить миграции и убедиться в корректности структуры БД.
5. Зарегистрировать модели в admin.py и настроить отображение связанных данных (list_display, list_filter, list_editable, search_fields).
6. Проверить работу через админ-панель: создать записи, связать их, проверить каскадные операции.

Вариант 1. Интернет-магазин: товары и категории

Компонент	**Описание**
Первичная модель	Category (название, slug, описание)
Вторичная модель	Product (название, цена, количество, описание, фото)
Тип связи	ForeignKey (одна категория — много товаров)
on_delete	PROTECT (запретить удаление категории, если в ней есть товары)
Дополнительно	Настроить list_filter по категории и list_editable для цены

Вариант 2. Блог: посты и теги

Компонент	Описание
Первичная модель	Tag (название, slug)
Вторичная модель	Post (заголовок, текст, дата, автор)
Тип связи	ManyToManyField (один пост — много тегов, один тег — много постов)

Компонент	Описание
Дополнительно	Настроить filter_horizontal для удобного выбора тегов в админке

Вариант 3. Библиотека: книги и авторы

Компонент	Описание
Первичная модель	Author (имя, фамилия, биография, фото)
Вторичная модель	Book (название, год издания, ISBN)
Тип связи	ForeignKey (один автор — много книг)
on_delete	CASCADE (при удалении автора удаляются его книги)
Дополнительно	Добавить related_name='books' для обратного доступа

Вариант 4. Университет: студенты и группы

Компонент	Описание
Первичная модель	Group (название, курс, специальность)
Вторичная модель	Student (имя, фамилия, номер зачётки, email)
Тип связи	ForeignKey (одна группа — много студентов)
on_delete	SET_NULL (при удалении группы студент остаётся, но без группы)
Дополнительно	null=True для поля группы, чтобы студент мог быть без группы

Вариант 5. Медиатека: фильмы и жанры

Компонент	Описание
Первичная модель	Genre (название, описание)
Вторичная модель	Movie (название, режиссёр, год, рейтинг, постер)
Тип связи	ManyToManyField (один фильм — много жанров)
Дополнительно	Настроить list_filter по жанрам и search_fields по названию

Вариант 6. Ресторан: блюда и категории меню

Компонент	Описание
Первичная модель	MenuCategory (название, порядок отображения)
Вторичная модель	Dish (название, описание, цена, вес, фото)
Тип связи	ForeignKey (одна категория — много блюд)
on_delete	PROTECT
Дополнительно	list_editable для цены и ordering по порядку отображения

Вариант 7. Социальная сеть: пользователи и профили

Компонент	Описание
Первичная модель	User (стандартная модель Django)
Вторичная модель	Profile (био, аватар, дата рождения, телефон)
Тип связи	OneToOneField (один пользователь — один профиль)
on_delete	CASCADE
Дополнительно	Автоматическое создание профиля через сигнал post_save

Вариант 8. Музыка: альбомы и песни

Компонент	Описание
Первичная модель	Album (название, год выпуска, обложка, исполнитель)
Вторичная модель	Song (название, длительность, номер трека)
Тип связи	ForeignKey (один альбом — много песен)
on_delete	CASCADE
Дополнительно	ordering по номеру трека, list_filter по году

Вариант 9. Новостной сайт: новости и категории (по образцу лабораторной)

Компонент	Описание
Первичная модель	Category (название, slug, описание)
Вторичная модель	News (заголовок, содержание, фото, дата)
Тип связи	ForeignKey
on_delete	PROTECT
Дополнительно	list_display с категорией, list_filter по категории

Вариант 10. Форум: темы и сообщения

Компонент	Описание
Первичная модель	Topic (название, автор, дата создания, закрыта ли)
Вторичная модель	Message (текст, автор, дата, прикреплённые файлы)

Компонент	Описание
Тип связи	ForeignKey (одна тема — много сообщений)
on_delete	CASCADE
Дополнительно	search_fields по тексту сообщения, list_filter по дате

Вариант 11. Образование: курсы и уроки

Компонент	Описание
Первичная модель	Course (название, описание, преподаватель, цена)
Вторичная модель	Lesson (название, содержание, длительность, номер)
Тип связи	ForeignKey (один курс — много уроков)
on_delete	CASCADE
Дополнительно	ordering по номеру урока, list_editable для длительности

Вариант 12. Медицина: врачи и пациенты

Компонент	Описание
Первичная модель	Doctor (имя, специализация, кабинет)
Вторичная модель	Patient (имя, дата рождения, телефон, диагноз)
Тип связи	ForeignKey (один врач — много пациентов)
on_delete	SET_NULL (при удалении врача пациент остаётся)
Дополнительно	null=True, list_filter по специализации

Вариант 13. Такси: водители и поездки

Компонент	Описание
Первичная модель	Driver (имя, телефон, номер машины, рейтинг)
Вторичная модель	Trip (адрес отправления, адрес назначения, стоимость, дата)
Тип связи	ForeignKey (один водитель — много поездок)
on_delete	CASCADE
Дополнительно	list_filter по водителю, search_fields по адресам

Вариант 14. Склад: товары и поставщики

Компонент	Описание
Первичная модель	Supplier (название, страна, телефон, email)
Вторичная модель	Product (название, артикул, цена, количество)
Тип связи	ManyToManyField (один поставщик — много товаров, один товар — от многих поставщиков)
Дополнительно	filter_horizontal в админке, search_fields по названию

Вариант 15. Платформа: пользователи и комментарии

Компонент	Описание
Первичная модель	Article (заголовок, текст, автор, дата)
Вторичная модель	Comment (текст, автор, дата, родительский комментарий)

Компонент	Описание
Тип связи	ForeignKey (одна статья — много комментариев) + самосвязь для ответов
on_delete	CASCADE
Дополнительно	Древовидная структура комментариев, related_name='replies'

Требования к отчёту

1. Титульный лист - название работы, ФИО, группа, вариант.
2. Цель работы.
3. Теоретическое обоснование - краткое описание типов связей в Django.
4. Код моделей - с комментариями, объясняющими выбор типа связи и параметров.
5. Скриншоты миграций - makemigrations и migrate.
6. Скриншоты структуры БД - поля и ограничения (внешний ключ).
7. Скриншоты админ-панели - список объектов, форма добавления, фильтры.
8. Вывод - что было сделано, какие трудности возникли, как они решены.

Критерии оценивания

Критерий	Балл
Корректное создание моделей с правильным типом связи	15
Правильная настройка параметров (on_delete, related_name, null, blank, default)	10
Корректное применение миграций	10
Настройка админ-панели (list_display, list_filter, list_editable, search_fields)	10

Критерий	Балл
Проверка работы связей через админку	5
Оформление отчёта	5
Итого	**55**

Шкала перевода баллов в оценку:

Оценка	Баллы
5 (отлично)	50-55
4 (хорошо)	42-49
3 (удовлетворительно)	33-41
2 (неудовлетворительно)	0-32